Proyecto Final – Desarrollo Web



Objetivo

Desarrollar una aplicación web completa (frontend + backend separados) que implemente autenticación con JWT u otro método de autenticación seguro y que integre manejo de colas para procesar tareas asíncronas. El proyecto debe simular un entorno de trabajo real, con división de roles en el equipo y responsabilidades claras.

Contexto del Proy ecto

La empresa ficticia **Galacticos S.A.** ha solicitado un sistema web que permita gestionar un proceso de negocio (ejemplo: pedidos, turnos, reservas, tickets, órdenes de trabajo).

- El frontend será una aplicación SPA (Single Page Application) que consuma un backend en Node.js mediante una API REST.
- El backend debe implementar un sistema de autenticación con JWT y un esquema de colas para procesar trabajos (envío de correos, generación de reportes, tareas pesadas, notificaciones, etc.).
- Los equipos deben organizarse con roles definidos, simulando un proyecto en una empresa real.

Roles en el equipo (1-3 integrantes)

1. **Arquitecto de Software (1)**
    - Define la arquitectura del proyecto (frontend/backend/colas).
    - Documenta el diseño de la base de datos y la comunicación entre módulos.
    - Supervisa la integración de los distintos componentes.
2. **Desarrolladores Backend (1)**
    - Implementan el API en Node.js con Express.
    - Configuran JWT (registro, login, refresh tokens, middleware).
    - Implementan el sistema de colas (BullMQ, RabbitMQ o similar).
    - Diseñan y administran la base de datos.
3. **Desarrolladores Frontend (1)**
    - Implementan el frontend como SPA (React, Vue o Angular).
    - Manejan el consumo de la API con tokens JWT.
    - Construyen formularios con validaciones y componentes reutilizables.
    - Muestran el estado de las colas y las respuestas del backend al usuario.

Requisitos Minimos

- **Autenticación:** Registro, login, refresh de tokens, logout, protección de rutas y roles.
- **Entidad de negocio:** CRUD completo de la entidad elegida (pedidos, turnos, tickets, etc.).

- **Colas:** Al menos 2 trabajos asíncronos (ejemplo: envío de correos, notificaciones, generación de reportes).
- **Validaciones y seguridad básica:** Manejo de errores, validación de formularios, medidas mínimas de seguridad (CORS, Helmet o similar).
- **Frontend:** SPA con navegación, formularios validados, feedback visual de acciones y estado de trabajos en cola.

Ponderación

Criterio	Descripción	Puntos
Arquitectura y Roles	División de responsabilidades, documentación del diseño de arquitectura y base de datos	3
Autenticación	Implementación segura de JWT, protección de rutas, manejo de sesiones y roles	4
Backend API	Endpoints RESTful completos, validación de datos, manejo correcto de errores	3
Colas	Implementación de colas con al menos 2 trabajos, reintentos y manejo de fallos	6
Frontend SPA	Interfaz clara, formularios validados, consumo de API, feedback de estado de trabajos	4
Integración y trabajo en equipo	Evidencia de colaboración en roles, coherencia entre frontend y backend	3
Seguridad básica y buenas prácticas	Uso de middleware de seguridad, estructura de proyecto ordenada, código limpio	2
Total		**25**

Stack sugerido

- **Backend**: Node.js + Express, JWT (access + refresh), Joi/Zod para validación.
- **Colas** (elige una):
  - BullMQ (Redis): fácil de levantar y excelente para trabajos asíncronos.
  - RabbitMQ: más “enterprise” (publish/subscribe, routing keys).
- **Frontend**: React/Vue/Angular. Recomendado React + React Router + React Query.
- **BD**: PostgreSQL / MySQL (o MongoDB si te conviene).
- **Correo**: Nodemailer o API (SendGrid/Mailgun).
- **Env**: .env + dotenv.

Documentación a Entregar

Documento de Arquitectura (máx. 10 páginas)

- **Portada:** Nombre del proyecto, integrantes, roles, curso y fecha.
- **Resumen Ejecutivo:** Descripción breve del sistema y sus objetivos.
- **Arquitectura General:** Diagrama de la solución (Frontend, Backend, Base de Datos, Colas).
- **Modelo de Datos:** Diagrama ER o esquema de colecciones.
- **Flujo de Autenticación:** Explicación del uso de JWT (o método elegido).
- **Flujo de Colas:** Explicación de los trabajos encolados (qué hace cada uno, reintentos, fallos).
- **Roles y Responsabilidades:** Qué hizo cada miembro del equipo (arquitecto, backend, frontend).
- **Buenas prácticas de seguridad:** Medidas aplicadas (hash de contraseñas, validación, etc.).

Manual Técnico

- **Estructura del proyecto:** carpetas de backend y frontend.
- **Instrucciones de instalación:** requisitos previos, cómo instalar dependencias, levantar el sistema y workers de colas.
- **Variables de entorno:** .env.example.
- **Endpoints principales de la API:** tabla con método, ruta, descripción, parámetros y ejemplo de respuesta.
- **Uso de colas:** explicación de cómo se encolan trabajos y cómo se procesan.

Manual de Usuario

- **Ingreso al sistema:** login, registro, roles.
- **Flujo principal de negocio:** cómo se crean pedidos/reservas/tickets según el proyecto.
- **Evidencia de colas:** cómo ver el estado de trabajos (ej. panel bull-board o endpoint /jobs/:id).
- **Ejemplo de uso en el frontend**: capturas de pantalla de la app funcionando.

ENTREGABLE

Entrega: Sábado 31 de Octubre del 2026.
Modalidad: Presencial
Distribución de presentación (Maximo 10 minutos)

Arquitectura

- Explicación breve del diagrama de arquitectura.
- Roles en el equipo y cómo se distribuyó el trabajo.

Demo técnica

- Login y registro de usuario.
- Ejemplo de CRUD (crear, listar, actualizar, eliminar).
- Ejemplo de cola en acción (ejemplo: crear pedido -> se genera email/report/pdf en background).
- Evidencia de reintentos o cola de fallos (si aplica).

Conclusiones

- Principales aprendizajes (autenticación, colas, separación frontend/backend).
- Retos técnicos encontrados.
- Qué mejorarían si tuvieran más tiempo.

IDEAS DE PROYECTOS

1. Sistema de Reservas de Citas Médicas
- **Entidad de negocio:** pacientes, doctores, citas.
- **Colas:**
  - Enviar correos/SMS de confirmación de cita.
  - Generar recordatorios automáticos 24 horas antes.
- **Extra:** manejo de roles (administrador, doctor, paciente).

2. Plataforma de Pedidos en Línea (mini e-commerce)
- **Entidad de negocio:** usuarios, productos, pedidos.
- **Colas:**
  - Procesamiento de pago simulado.
  - Envío de confirmación de pedido con PDF de factura/ticket.
- **Extra:** inventario con actualización asíncrona.

3. Sistema de Tickets de Soporte Técnico
- **Entidad de negocio:** clientes, tickets, técnicos**.**
- **Colas:**
  - Asignación automática de tickets según prioridad.
  - Notificación de seguimiento (correo/SMS).
- **Extra:** vista de tablero con estados (pendiente, en progreso, resuelto).

4. Aplicación de Reservas de Eventos
- **Entidad de negocio:** usuarios, eventos, reservas.
- **Colas:**
  - Generación de entrada en PDF con QR.
  - Envío de confirmación por correo.
- **Extra:** control de aforo y validación en tiempo real.

5. Gestión de Turnos en un Banco
- **Entidad de negocio:** clientes, turnos, cajeros**.**
- **Colas:**
  - Asignar turno en cola virtual y notificar al cliente.
  - Alertar al cajero cuando un turno esté próximo.
- **Extra:** panel en frontend con el estado de la cola en vivo.

6. Sistema de Reservas de Transporte Bus
- **Entidad de negocio:** usuarios, rutas, reservas.
- **Colas:**
  - Confirmación de reserva + envío de boleto digital.
  - Notificación previa a la salida del viaje.
- **Extra:** cálculo de ocupación en tiempo real.